

Pacman Game

Master 1 Artificial Intelligence

Abdelbaki Kacem

M1 AI

Abou Orm Daniel

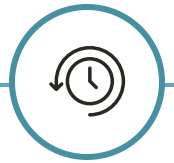
M1 AI

Kabban Joe

M1 AI

Presentation Outline

An overview of the topics we will cover



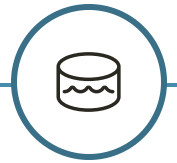
Phase 1 Recap

A brief overview of our previous progress.



Phase 2 - New Features

Introduction to the latest enhancements.



System Architecture

Understanding the underlying structure.



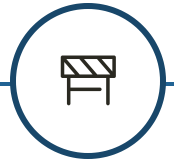
Ghosts & AI

Exploring the artificial intelligence of our characters.



Cloud & DevOps

Our approach to infrastructure and deployment.



Challenges Encountered

Lessons learned and obstacles overcome.

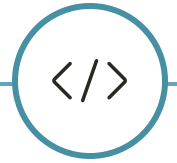


Demo & Conclusion

A live demonstration followed by closing remarks.

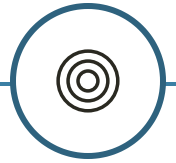
Phase 1 Recap

Key Accomplishments and Next Steps



Maze Implementation

Successfully implemented the maze generation algorithms in Python DFS, Tetris, Kruskal



Objective Defined for Pac-Man Maze

Set goal to create a Pac-Man optimized maze with symmetry, a ghost room, and teleportation tunnels.



CI/CD Setup

Established GitHub Actions for testing, Docker builds, and deployment on Render.com.

Phase 2: New Features

Key Enhancements and Functionality

Full GUI

Developed a desktop client using Java/LibGDX with Pac-Man rendering.

MongoDB Atlas Persistence

Utilized MongoDB Atlas for storing mazes, ratings, and replays.

Ghost AI

Introduced 4 ghosts with distinct individual behaviors and modes.

Difficulty Levels

Options for Classic (Greedy) and A* (Optimal Pathfinding) difficulty levels.

Player AI Bot

AI bot strategies including GREEDY and GHOST_AVOIDANCE.

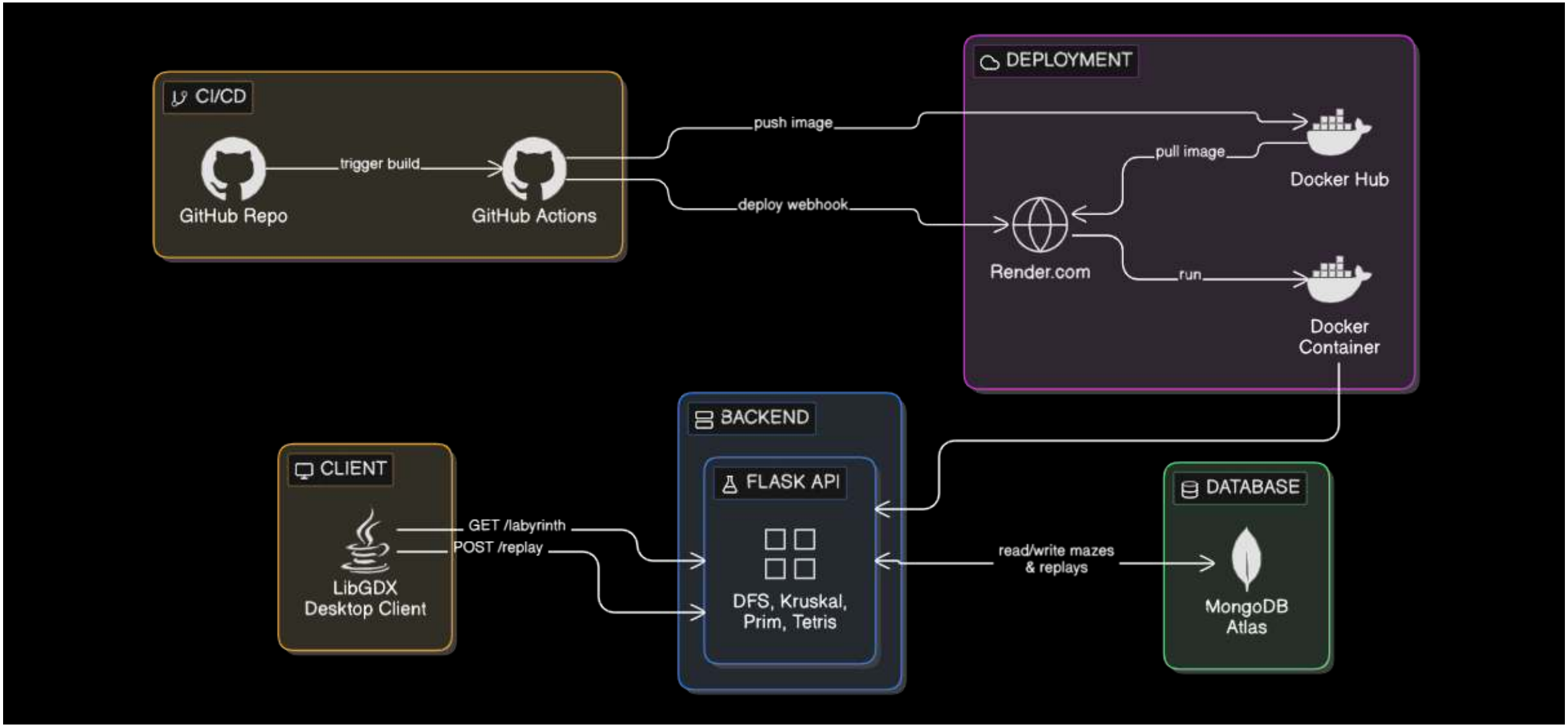
Race Mode

Added a competitive race mode for players.

Replay System

Functionality to record and playback completed games.

System Architecture



BACK END

Swagger's Labyrinth APIs

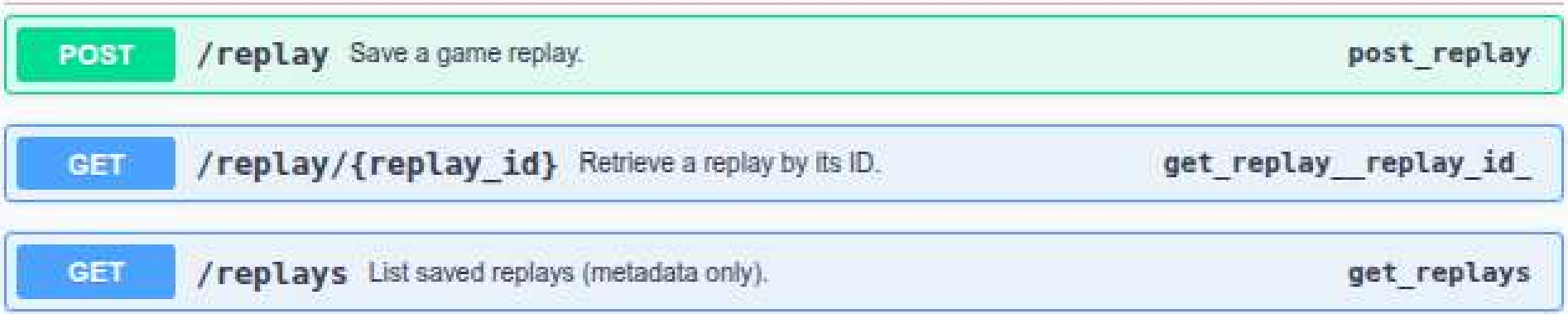
Maze generation and retrieval

GET	`/labyrinth`	Generate a maze and save it to the database.	`get_labyrinth`
GET	`/labyrinth/{maze_id}`	Retrieve a maze by its ID.	`get_labyrinth__maze_id_`
GET	`/mazes`	List all saved mazes (metadata only).	`get_mazes`

Maze rating


POST
/labyrinth/rate
Rate a maze by its ID.
post_labyrinth_rate
Parameters
Try it out
Name
Description
body
required
object
(body)
Example Value
Model
{
"maze_id": "string",
"rating": 0
}
Parameter content type
application/json

Game replay management



MongoDB Implementation

Cloud-Hosted Atlas for Efficient Data Management



- ## Cloud-Hosted MongoDB Atlas

  Utilizing the 'pacmanTer' cluster for reliable data management, accessed securely via the SRV protocol with `pymongo`.

- ## Key Collections

  Storing maze data in 'mazes' (algorithm, dimensions, ratings) and game sessions in 'replays' (states, outcomes).

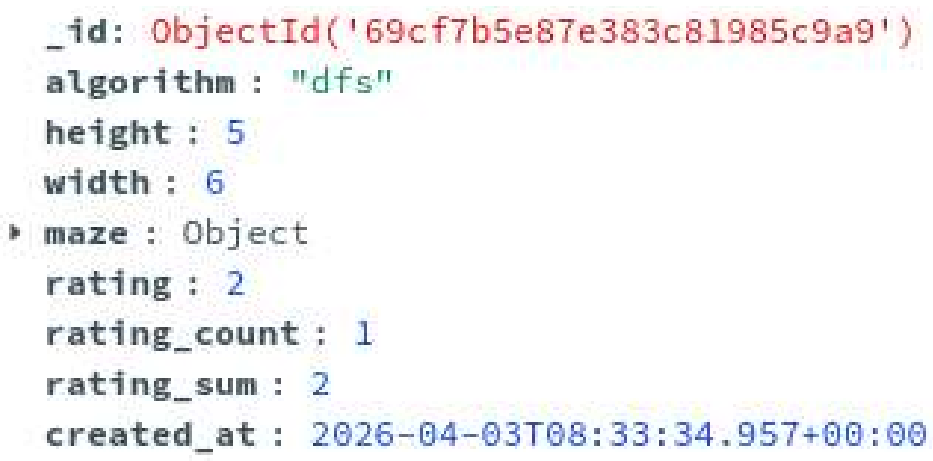
- ## Resilience and Integrity

  Ensuring graceful degradation for database operations using try-except blocks and maintaining integrity through document references and insert validation.

- ## Tech stack

  Python Flask + pymongo[srv] driver connecting to MongoDB Atlas via SRV protocol, deployed with Docker

Maze table entry



Replay table entry



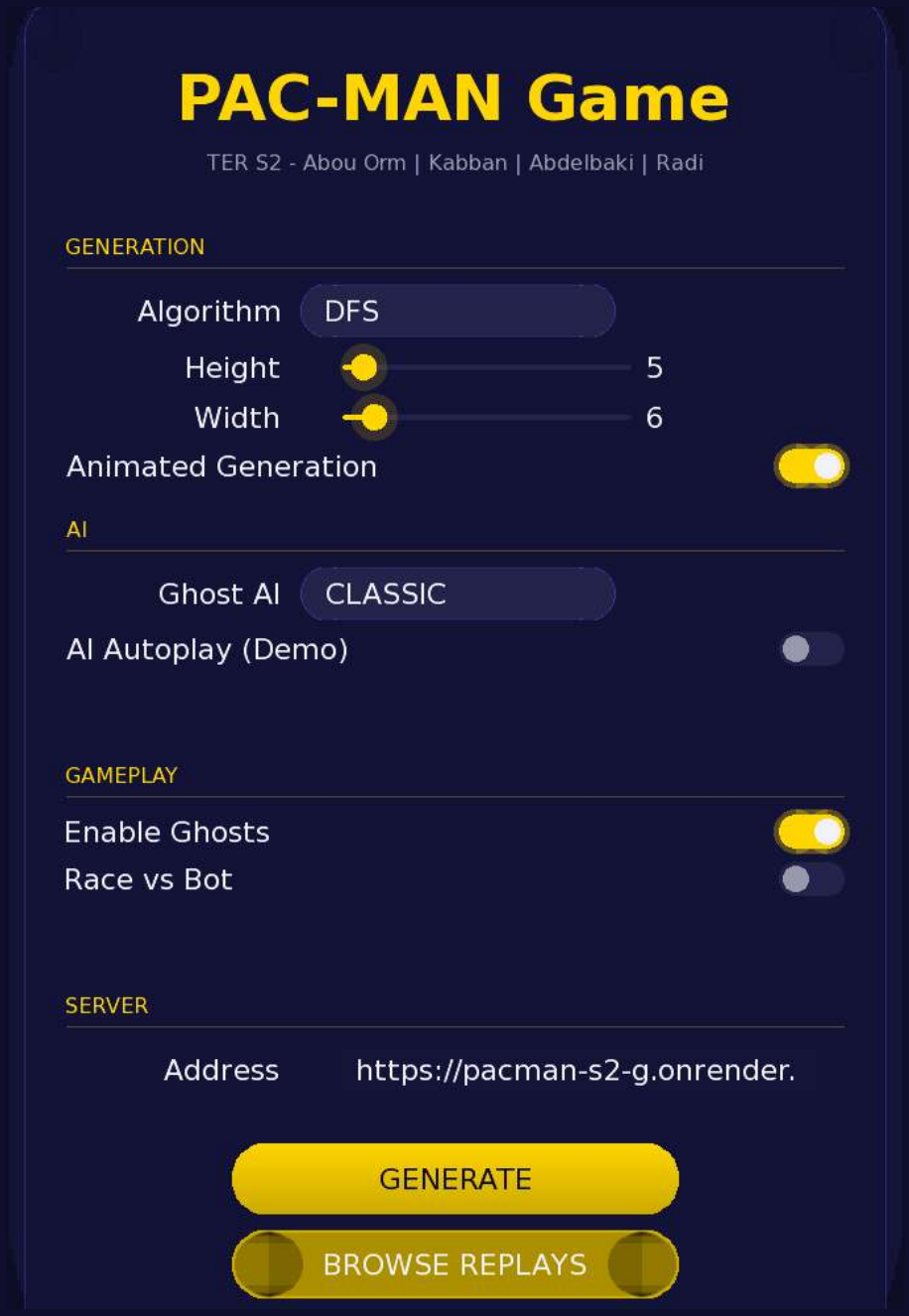
MongoDB Data Model & Architecture

Collections

- **`mazes` Collection**
  - Algorithm used (e.g., DFS, Prim's)
  - Dimensions (width, height)
  - Full maze structure data
  - Community rating system (average, count, sum)
- **`replays` Collection**
  - Player type (human/bot)
  - Final score
  - Dots eaten, win/loss status
  - Game duration
  - Frame-by-frame game state and player inputs

FRONT-END

MENU SCREEN


PAC-MAN Game
TER S2 - Abou Orm | Kabban | Abdelbaki | Radi
GENERATION
Algorithm
DFS
Height
5
Width
6
Animated Generation
AI
Ghost AI
CLASSIC
AI Autoplay (Demo)
GAMEPLAY
Enable Ghosts
Race vs Bot
SERVER
Address
https://pacman-s2-g.onrender.
GENERATE
BROWSE REPLAYS

Browse Replays menu



DEMO-REPLAY


< Back
SAVED REPLAYS
Found 33 replay(s).
WATCH
DFS (11x5) - HUMAN
WATCH
DFS (11x5) - HUMAN
WATCH
DFS (11x5) - BOT
WATCH
DFS (11x5) - BOT
Score: 2370 | Result: WON | 2026-04-15 11:57:11.912000
WATCH
DFS (11x5) - BOT
WATCH
DFS (11x5) - BOT
Score: 770 | Result: WON | 2026-04-09 15:20:32.037000
WATCH
DFS (11x5) - BOT
Score: 770 | Result: WON | 2026-04-08 21:11:32.951000
WATCH
DFS (11x5) - BOT
Score: 1970 | Result: WON | 2026-04-08 21:10:53.555000
WATCH
DFS (11x5) - HUMAN
WATCH
DFS (31x11) - HUMAN
WATCH
DFS (21x15) - HUMAN
WATCH

The Four Apparitions

Individual Ghost Behaviors in Pac-Man

Blinky (Red)



Directly targets Pac-Man's position (Aggressive).

Pinky (Pink)



Targets a position 4 tiles ahead of Pac-Man (Ambusher).

Inky (Cyan)



Target position is calculated based on Blinky and Pac-Man (Unpredictable).

Clyde (Orange)



Chases when far, scatters when close (Timid).

Blinky



```java
public class Blinky extends Ghost {

    public Blinky(MazeData maze, int startRow, int startCol) {
        super(maze, startRow, startCol, 0, maze.width - 1);
    }

    @Override
    public int[] getTargetTile(int pacmanRow, int pacmanCol, int pacmanDir) {
        switch (mode) {
            case CHASE:
                return new int[]{pacmanRow, pacmanCol};
            case SCATTER:
                return new int[]{scatterRow, scatterCol};
            case FRIGHTENED:
                // Random target — handled by caller via random direction
                return new int[]{scatterRow, scatterCol};
            case EATEN:
                // Return to ghost house center
                return new int[]{maze.height / 2, maze.width / 2};
            default:
                return new int[]{pacmanRow, pacmanCol};
        }
    }
}
```

Pinky



```java
public int[] getTargetTile(int pacmanRow, int pacmanCol, int pacmanDir) {
    switch (mode) {
        case CHASE:
            // 4 tiles ahead of Pac-Man
            int targetR = pacmanRow + DR[pacmanDir] * 4;
            int targetC = pacmanCol + DC[pacmanDir] * 4;
            // Clamp to maze bounds
            targetR = Math.max(0, Math.min(maze.height - 1, targetR));
            targetC = Math.max(0, Math.min(maze.width - 1, targetC));
            return new int[]{targetR, targetC};
        case SCATTER:
            return new int[]{scatterRow, scatterCol};
        case FRIGHTENED:
            return new int[]{scatterRow, scatterCol};
        case EATEN:
            return new int[]{maze.height / 2, maze.width / 2};
        default:
            return new int[]{pacmanRow, pacmanCol};
    }
}
```

Inky



```java
public int[] getTargetTile(int pacmanRow, int pacmanCol, int pacmanDir) {
    switch (mode) {
        case CHASE:
            // 2 tiles ahead of Pac-Man
            int aheadR = pacmanRow + DR[pacmanDir] * 2;
            int aheadC = pacmanCol + DC[pacmanDir] * 2;
            // Vector from Blinky to that position, doubled
            int targetR = aheadR + (aheadR - blinky.getGridRow());
            int targetC = aheadC + (aheadC - blinky.getGridCol());
            targetR = Math.max(0, Math.min(maze.height - 1, targetR));
            targetC = Math.max(0, Math.min(maze.width - 1, targetC));
            return new int[]{targetR, targetC};
        case SCATTER:
            return new int[]{scatterRow, scatterCol};
        case FRIGHTENED:
            return new int[]{scatterRow, scatterCol};
        case EATEN:
            return new int[]{maze.height / 2, maze.width / 2};
        default:
            return new int[]{pacmanRow, pacmanCol};
    }
}
```

Clyde



```
c int[] getTargetTile(int pacmanRow, int pacmanCol, int pacmanDir) {
witch (mode) {
    case CHASE:
        double dist = Math.sqrt(
            Math.pow(gridRow - pacmanRow, 2) + Math.pow(gridCol - pacmanCol
        );
        if (dist > FLEE_DISTANCE) {
            return new int[]{pacmanRow, pacmanCol};
        } else {
            return new int[]{scatterRow, scatterCol};
        }
    case SCATTER:
        return new int[]{scatterRow, scatterCol};
    case FRIGHTENED:
        return new int[]{scatterRow, scatterCol};
    case EATEN:
        return new int[]{maze.height / 2, maze.width / 2};
    default:
        return new int[]{pacmanRow, pacmanCol};
```

Ghost AI: Modes of Operation



SCATTER

Ghosts momentarily retreat to their designated corner of the maze, offering a brief respite before returning to the chase.



CHASE

Ghosts actively pursue Pac-Man using their unique targeting strategies. This is the primary offensive state.



FRIGHTENED

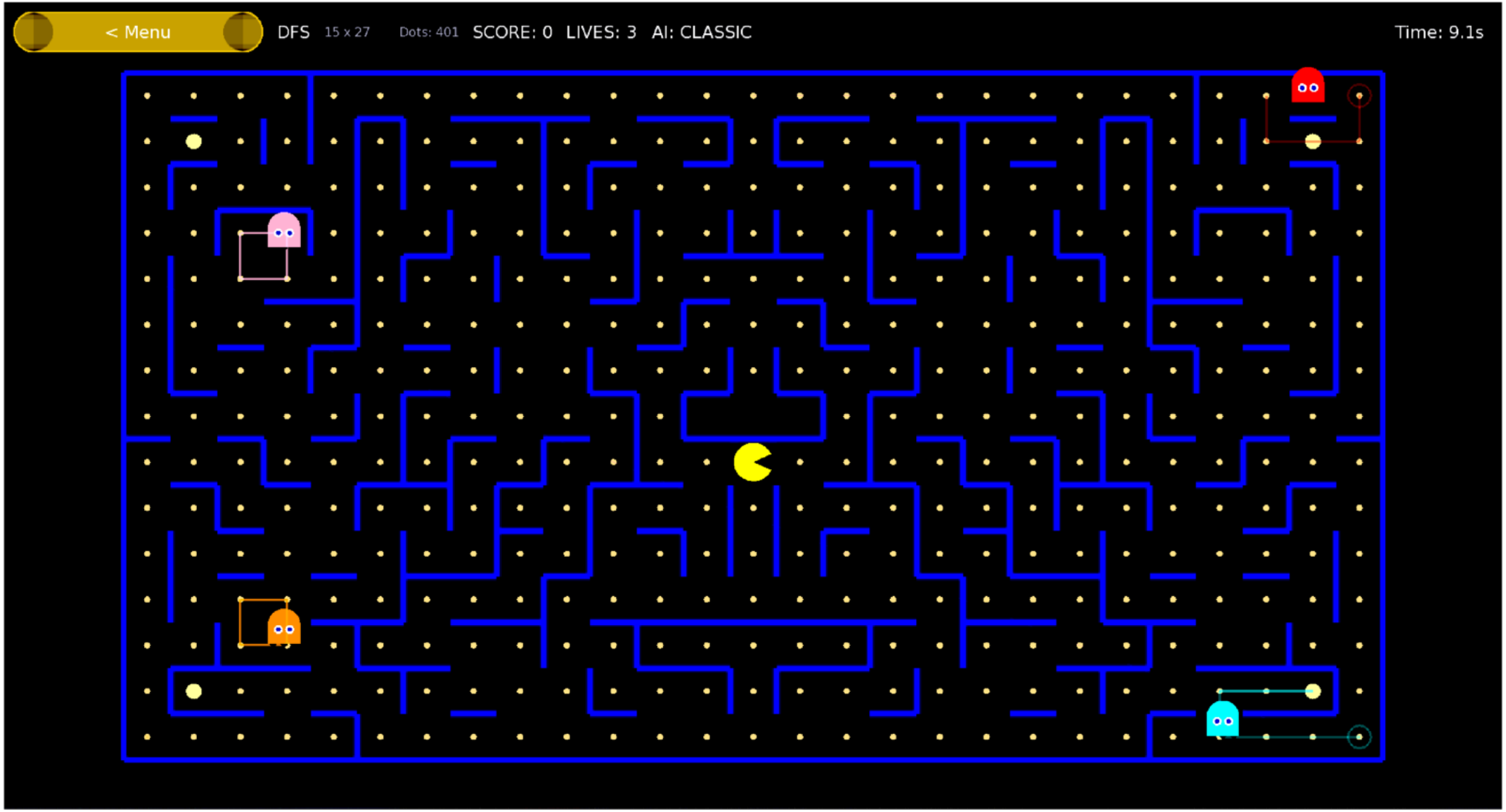
Triggered by a Power Pellet. Ghosts turn blue, move randomly, and attempt to evade Pac-Man. Vulnerable to being eaten.



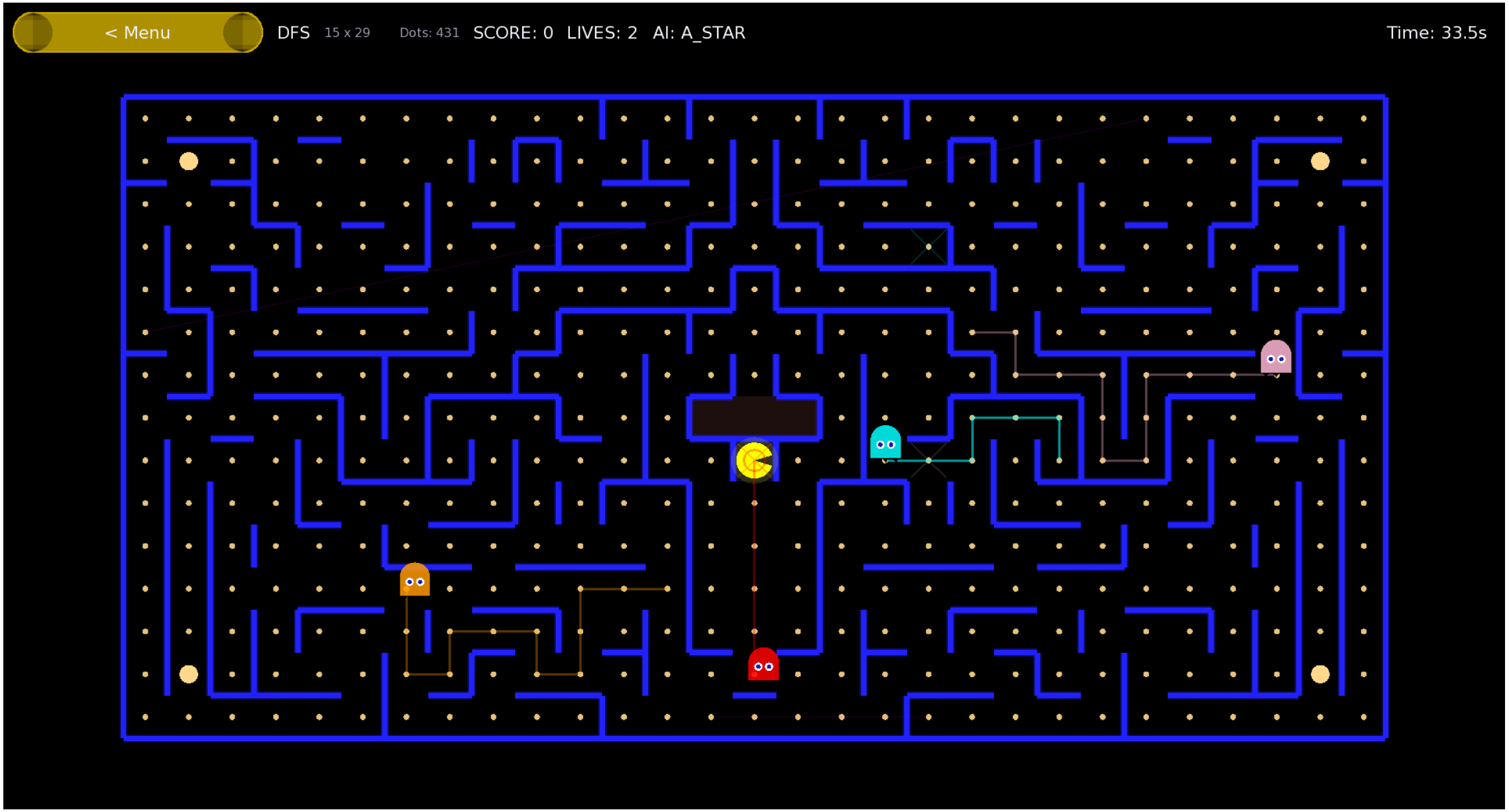
EATEN

After being consumed by Pac-Man, ghosts return to the ghost pen as eyes, passing through walls to respawn.

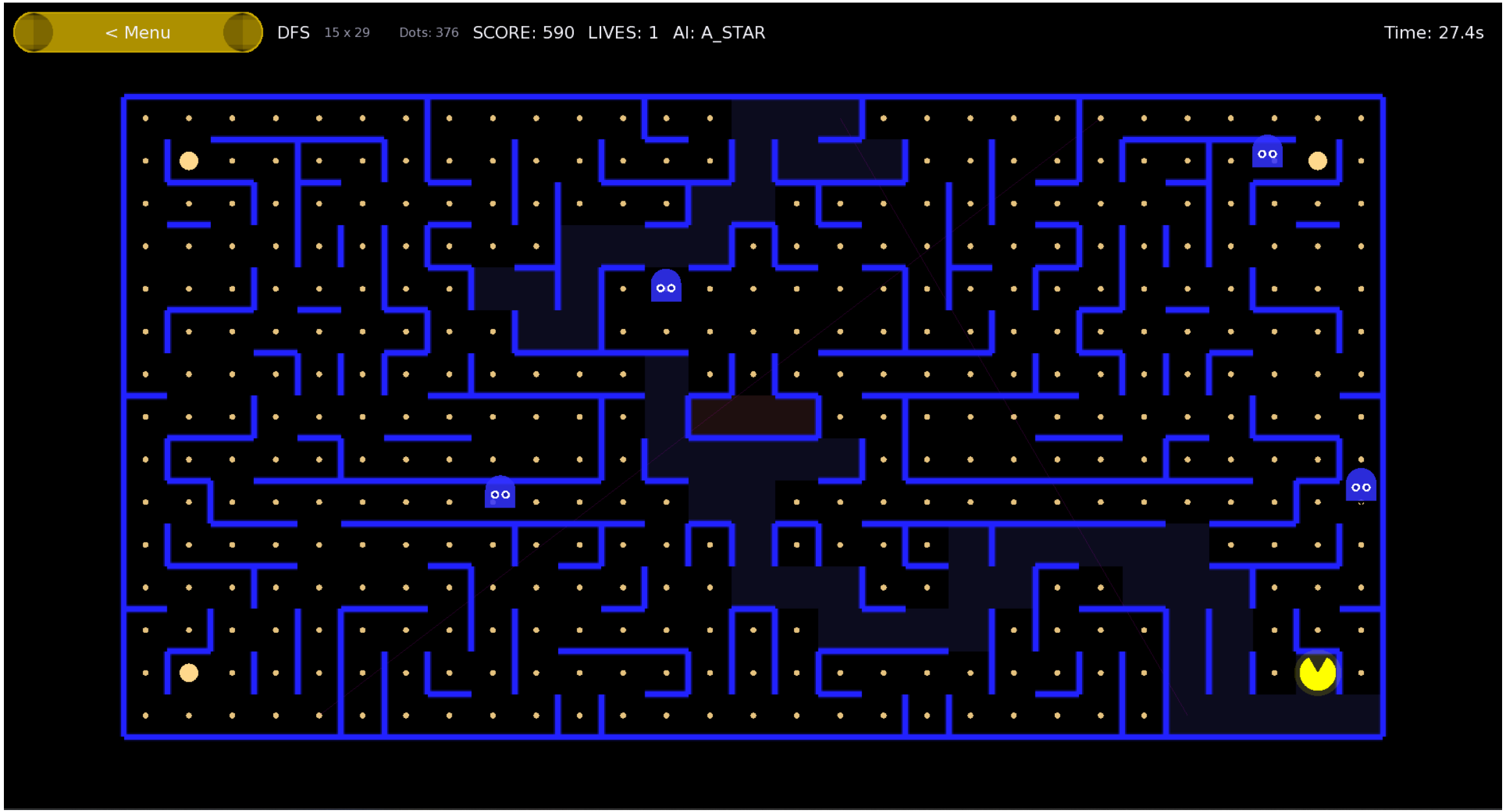
Scatter mode(*Visiting corners*)



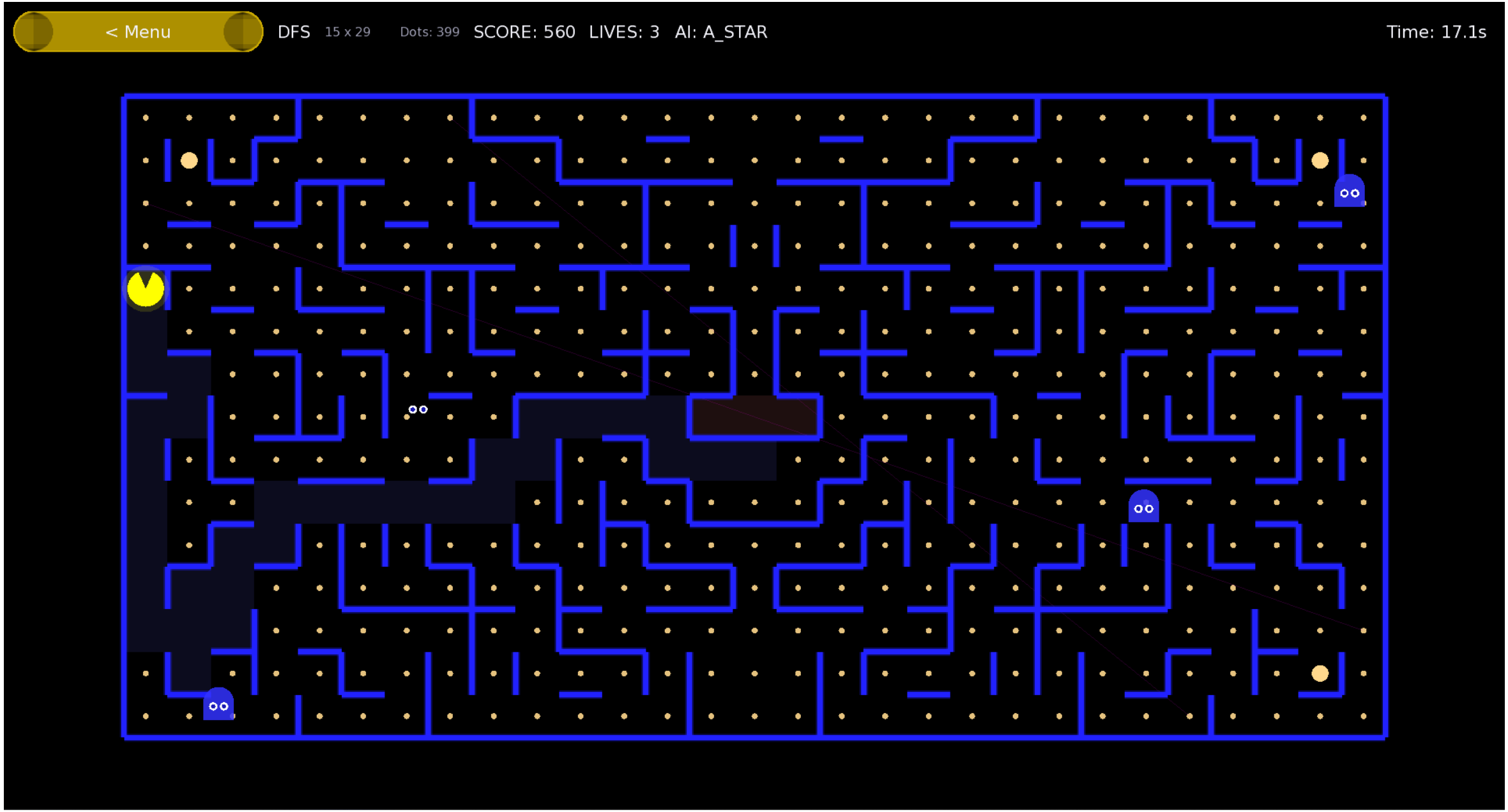
Chase mode(*Targeting pacman*)



Frightened mode(*escaping pacman*)



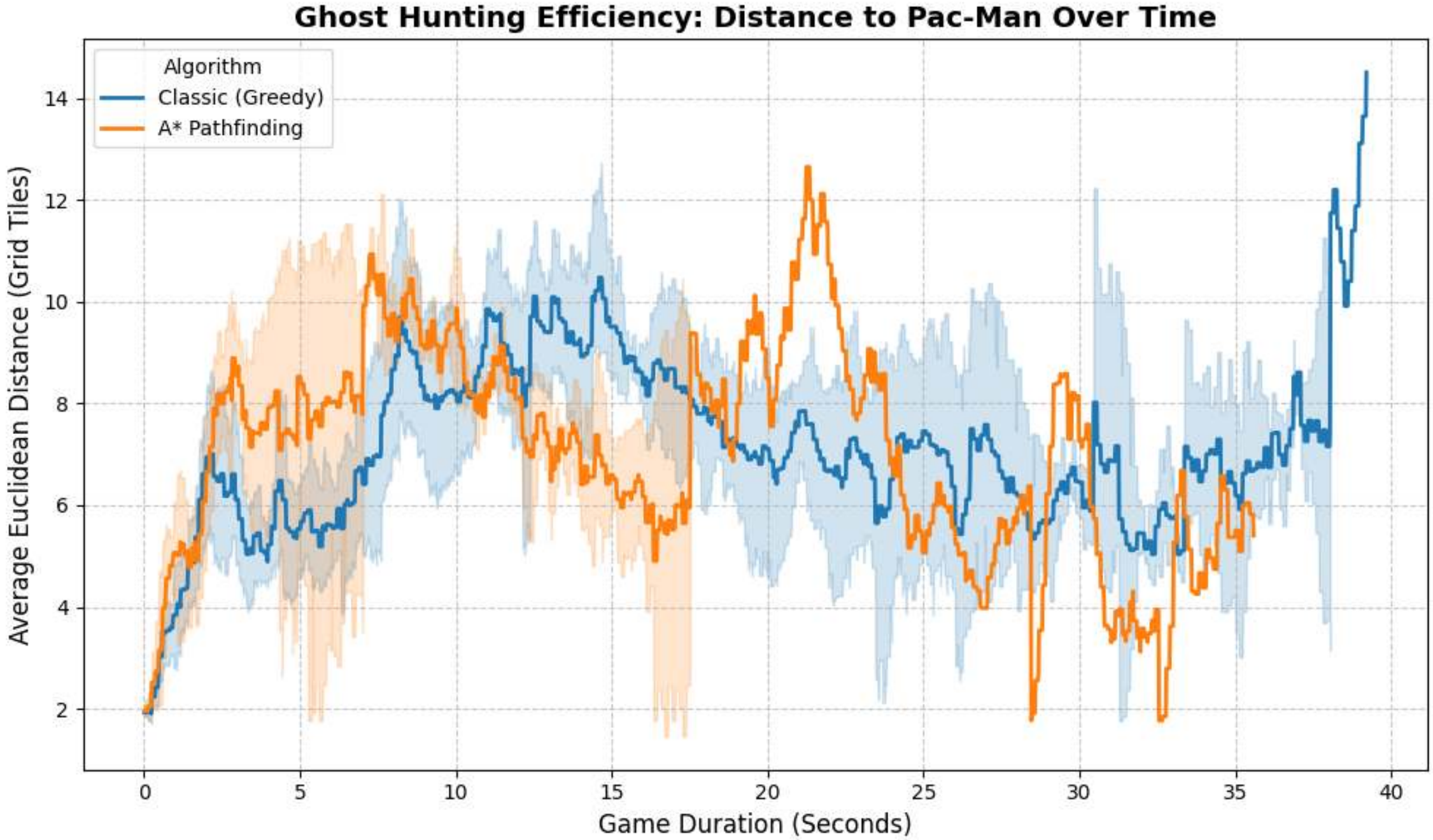
Eaten mode(*Going back to base*)



Ghost AI: Difficulty Levels

Comparing Pursuit and Navigation Strategies

Feature	Classic (Greedy)	A* (A-Star)
Algorithm	Greedy Best-First Search	A* Search (Optimal Pathfinding)
Heuristic/Metric	Squared Euclidean Distance	Manhattan Distance (Heuristic)
Exploration	Evaluates immediate neighbors only	Explores the complete maze graph
Data Structure	Directional priority (Up ; Left ; Down ; Right)	Priority Queue for exploration
Behavior	Fast but potentially suboptimal	Finds the most optimal path


Ghost Hunting Efficiency: Distance to Pac-Man Over Time
Algorithm
Classic (Greedy)
A* Pathfinding
Average Euclidean Distance (Grid Tiles)
Game Duration (Seconds)
2
4
6
8
10
12
14
0
5
10
15
20
25
30
35
40

Player AI Bot Strategies

- **GREEDY Bot** 🎯
  - Utilizes Breadth-First Search (BFS) to locate the closest dot.
  - Prioritizes speed, disregarding ghost proximity.

- **GHOST_AVOIDANCE Bot** 👻
  - Employs a weighted Dijkstra algorithm.
  - Increases cell costs near ghosts to balance dot collection with evasion.



GREEDY ALGO

```java
public Direction getGreedyMove() {
    Queue<Tile> queue = new LinkedList<>(); // Standard BFS Queue
    queue.add(pacmanLocation);

    while (!queue.isEmpty()) {
        Tile current = queue.poll();

        // Check surrounding tiles (Up, Down, Left, Right)
        for (Tile neighbor : current.getValidNeighbors()) {

            // ✅ We found food! Immediately trace back the first turn to take
            if (neighbor.hasDot()) {
                return neighbor.getFirstTurnDirection();
            }

            // Otherwise, keep analyzing outwards
            if (!neighbor.isVisited()) {
                neighbor.setVisited(true);
                queue.add(neighbor);
            }
        }
    }
    return DEFAULT_DIRECTION;
}
```

ghost_avoidance
Algo

```
public Direction getSafeMove() {
    DangerMap danger = buildGhostHeatMap();
    PriorityQueue<Tile> pq = new PriorityQueue<>(sortByLowestCost());

    while (!pq.isEmpty()) {
        Tile current = pq.poll();
        if (current.hasFood()) return current.getFirstTurn();

        for (Tile next : current.getNeighbors()) {
            // MATH PENALTY: Cost increases heavily if ghost is near!
            float stepCost = 1.0f + (danger.getLevel(next) * 3.0f);

            if (current.getCost() + stepCost < next.getCost()) {
                next.setCost(current.getCost() + stepCost);
                pq.add(next);
            }
        }
    }
}
```

Demo & Conclusion

Key Takeaways and Next Steps



Live Demo / Video Showcase

Interactive demonstration of the project's capabilities.